

QUICKSORT

Divide, Conquer, and Outperform

codeHive

1. The Speed King

Quicksort lives up to its name. It is one of the most efficient sorting algorithms ever created. It uses a strategy called **Divide and Conquer** to break a massive sorting task into tiny, manageable pieces.

Core Principle: The Pivot

At the heart of Quicksort is the **Pivot**. The algorithm picks one value from the array and reorders all other elements so that:

- Values **smaller** than the pivot go to its left.
- Values **larger** than the pivot go to its right.

This process is called **Partitioning**.

2. Manual Run-Through

Initial Unsorted Array: [11, 9, 12, 7, 3]

STEP 1 Choose **3** as the pivot (last element).



STEP 2 Compare others to 3. Since all are larger, 3 must move to the front. Swap 3 with 11.



Note: "3" is now locked in its final sorted position.

STEP 3 Now sort the sub-array [9, 12, 7, 11]. Choose **11** as pivot.



STEP 4 Reorder around 11. Move 7 to the left and 12 to the right. Swap 11 and 12.



3. Power of Recursion

Quicksort doesn't just stop after one pivot. It calls itself (recursion) on the left side and the right side of the pivot. This continues until the sub-arrays are only 1 element long.

The Recursive Flow:

1. Partition the array.
2. `quicksort(left_side)`
3. `quicksort(right_side)`

Because the array keeps getting split in half, the number of steps required to sort doesn't double when the data doubles—it grows much slower!

4. Python Implementation

The code consists of two functions: the **partition** logic and the **recursive** driver.

```
def partition(array, low, high):
    pivot = array[high]
    i = low - 1

    for j in range(low, high):
        if array[j] ≤ pivot:
            i += 1
            array[i], array[j] = array[j], array[i]

    array[i+1], array[high] = array[high], array[i+1]
    return i+1

def quicksort(array, low=0, high=None):
    if high is None:
        high = len(array) - 1

    if low < high:
        pivot_index = partition(array, low, high)
        quicksort(array, low, pivot_index-1)
        quicksort(array, pivot_index+1, high)
```

5. Efficiency Analysis

Average Case: The Ideal Scenario

$$O(n \log n)$$

On average, Quicksort is incredibly efficient. If the pivot splits the array roughly in half each time, we achieve logarithmic depth.

Worst Case: The Warning

$$O(n^2)$$

If the pivot always picks the smallest or largest element (which happens if the array is **already sorted** and we pick the last element), Quicksort performs poorly, behaving like Bubble Sort.

6. Code Challenge

What are the missing recursive calls to make this function work?

```
pivot_index = partition(array, low, high)
_____(array, low, pivot_index-1)
_____(array, pivot_index+1, high)
```

Answer: quicksort

© 2026 codeHive Academy • Mastering Quicksort